

[Name of Your School / Department]

[University Name]

PROJECT REPORT

Expense Tracker

A Menu-Driven Command-Line Expense Management
Application in Python

Submitted as part of the **VITyarthi – Build Your Own Project** evaluation

Course: [Course Name] ([Course Code])

Submitted by	[Your Full Name]
Registration Number	[Your Registration Number]
Submitted to	[Faculty Name]
Slot	[Slot]
Date of Submission	[DD Month YYYY]
GitHub Repository	https://github.com/1divyansh/Expense-tracker-

Table of Contents

1. Introduction	2
2. Problem Statement	3
3. Functional Requirements	4
4. Non-Functional Requirements	5
5. System Architecture	6
6. Design Diagrams	7
7. Design Decisions & Rationale	12
8. Implementation Details	13
9. Screenshots / Results	17
10. Testing Approach	24
11. Challenges Faced	26
12. Learnings & Key Takeaways	27
13. Future Enhancements	28
14. References	29

List of Figures

Figure 1: System Architecture	6
Figure 2: Use Case Diagram	7
Figure 3: Workflow Diagram	8
Figure 4: Sequence Diagram for Adding an Expense	9
Figure 5: Component Diagram	10
Figure 6: Program start and main menu	17
Figure 7: Adding an expense	18
Figure 8: Viewing all expenses	19
Figure 9: Category-wise summary	19
Figure 10: Budget exceeded warning	20
Figure 11: Deleting an expense with confirmation	21
Figure 12: Handling invalid menu and category input	22
Figure 13: Exit summary	23
Figure 14: Git commit history	23

1. Introduction

Managing day-to-day spending is a common problem, especially for students who live on a fixed monthly allowance. Small expenses such as snacks, travel and subscriptions add up quickly, and without a record it is hard to know where the money went or whether the month's budget has already been crossed.

Expense Tracker is a menu-driven command-line application written in Python that helps a user record expenses, group them into categories, see a category-wise summary, and compare total spending against a budget. The user works through a simple numbered menu, so no technical knowledge is needed.

The project applies core programming concepts from the course:

- **Functions and modular design:** each feature is its own function, and expense creation is kept in a separate file (`expense.py`)
- **Data structures:** a *list* holds all expenses, each expense is a *dictionary*, and a dictionary is also used to group totals by category
- **Control flow:** a `while` loop drives the menu, `if/elif` chooses the action, and `for` loops process the data
- **User input and formatted output:** `input()`, type conversion with `int()/float()`, and f-strings
- **Version control:** the project was developed and tracked using Git and GitHub

1.1 Technologies and Tools Used

Item	Details
Programming language	Python 3 (version 3.6 or newer)
Libraries	None. Only Python built-in functions are used.
Interface	Command-line interface (terminal)
Version control	Git, with the repository hosted on GitHub
Editor / IDE	[e.g. VS Code / PyCharm / IDLE]
Diagrams	Mermaid (https://mermaid.js.org); source files in <code>diagrams/</code>

2. Problem Statement

People, and students in particular, often do not keep track of their daily expenses. As a result they:

- do not know how much they have spent in total,
- cannot tell which category (food, home, entertainment and so on) takes most of their money, and
- find out they have gone over budget only when the money has run out.

Spreadsheets and finance apps exist, but they can feel heavy for someone who just wants to note down a few expenses quickly. There is a need for a **lightweight, easy-to-use tool** that records expenses, organises them by category, and warns the user when spending crosses a budget.

2.1 Objectives

1. Let the user **add expenses** with a name, an amount and a category.
2. Let the user **view** and **delete** recorded expenses.
3. Show a **category-wise summary** with totals, percentages and a simple text bar chart.
4. Let the user **set a budget** and show the remaining amount, with a **warning** when spending goes over it.
5. Keep the program **simple, fast and dependency-free**, so it runs on any computer that has Python installed.
6. Apply course concepts (functions, lists, dictionaries, loops, conditionals, input/output) in a clean, modular way.

2.2 Scope

In scope:

- Adding, viewing and deleting expenses during one run of the program
- Five fixed categories: Food, Home, Work, Entertainment, Misc
- Category-wise summary and budget tracking
- Command-line menu interface

Out of scope (current version):

- Saving data after the program closes (data is kept in memory only)
- Dates, monthly reports or multiple users
- A graphical user interface

2.3 Target Users

- Students managing a monthly allowance
- Individuals who want a quick, simple way to track daily spending
- Beginner programmers who want to study a small but complete Python project

3. Functional Requirements

The system is organised into **five functional modules**.

ID	Module	Requirement	Input	Output
FR-1	Expense Entry	The user can add an expense with a name, an amount and a category chosen from a numbered list.	Name (text), amount (number), category number (1–5)	Confirmation such as <code>Saving the Expense: Rent (Home) - 1200.0</code>
FR-2	Expense Listing	The user can view all recorded expenses as a numbered list.	Menu choice <code>2</code>	Numbered list, or <code>[] No expenses recorded</code> if empty
FR-3	Expense Deletion	The user can delete an expense by its number after confirming. Pressing Enter without a number cancels.	Expense number, then <code>y / n</code>	<code>Deleted ...</code> or <code>Cancelled.</code>
FR-4	Reporting / Analytics	The user can view a summary of spending per category with amount, percentage and a text bar chart, plus the grand total.	Menu choice <code>4</code>	Category-wise table and <code>Total spent</code>
FR-5	Budget Management	The budget starts at Rs.5000 and the user can change it. After adding, viewing, deleting or summarising, the system shows the budget, the amount spent, and either the remaining amount or an over-budget warning.	New budget (whole number)	<code>[] budget: Rs.X, spent: Y</code> and <code>Remaining Budget</code> or <code>WARNING</code>
FR-6	Session Control	The main menu repeats until the user exits. An invalid menu choice shows an error and the menu again. On exit, the number of expenses and the total spent are shown.	Menu choice <code>1–6</code>	Menu, error message, or exit summary

3.1 Input / Output Structure

- **Input:** all input is typed at the keyboard in response to clear prompts, for example `Enter your choice [1 - 6]:` and `Enter expense amount: .`
- **Output:** all output is plain text in the terminal. Sections are separated by lines of dashes, and status messages start with `[]` so they stand out.

3.2 User Workflow

1. Start the program. The main menu appears.
2. Choose **1** to add expenses (repeat as needed).
3. Choose **2** to view them, or **4** to see the category summary.
4. Choose **5** to change the budget if needed. Budget status is shown after each action.
5. Choose **3** to delete a wrong entry.
6. Choose **6** to exit and see the session summary.

4. Non-Functional Requirements

ID	Category	Requirement	How it is met
NFR-1	Usability	The app must be usable without any training.	Numbered menus, the valid range shown in every prompt (e.g. [1 - 6]), clear confirmations, and a y/n question before deleting.
NFR-2	Performance	Every action should respond instantly.	All data is in memory. Adding is O(1); viewing, totals and summary are O(n) single passes over the list.
NFR-3	Portability	The app must run on Windows, macOS and Linux.	Pure Python 3 with no third-party libraries and no platform-specific code.
NFR-4	Maintainability	Code should be easy to read and extend.	One function per feature, a separate <code>expense.py</code> for creating expenses, shared helpers (<code>expense_msg</code> , <code>total_expenses</code>) instead of repeated code, and history tracked in Git.
NFR-5	Resource Efficiency	The app should use very little memory and disk.	Only a list of small dictionaries is stored. No files, database or background processes.
NFR-6	Error Handling	Invalid choices must not break the menu.	Invalid menu choices and out-of-range category numbers show a message and ask again. (<i>Known limitation: non-numeric numbers are not handled yet; see Sections 10 and 13.</i>)
NFR-7	Privacy / Security	The user's financial data must not leak.	Nothing is written to disk or sent over a network. All data is cleared when the program ends.

5. System Architecture

The application follows a simple **layered architecture** inside a single process:

1. **Presentation layer:** the `main()` function shows the menu, reads the user's choice and calls the right feature function.
2. **Feature (logic) layer:** one function per feature: `get_expenses`, `view_expenses`, `delete_expense`, `summrize`, `budget_status` and `exit`.
3. **Helper layer:** small reusable functions: `choose_expense`, `expense_msg` and `total_expenses`.
4. **Model:** `NewExpense()` in `expense.py` builds a single expense record (a dictionary).
5. **Data (in memory):** the `expenses` list and the `budget` value, both created in `main()` and passed to the functions that need them.

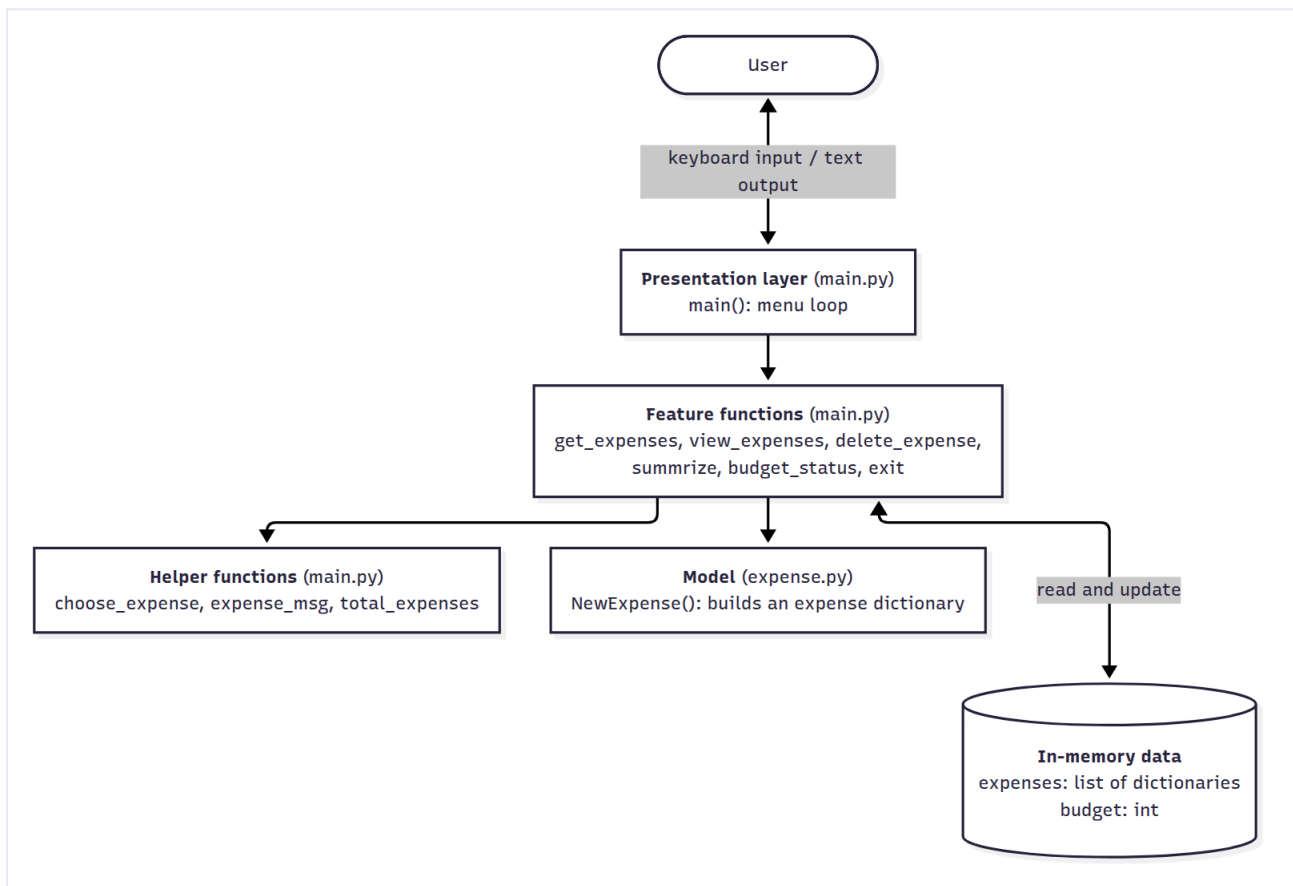


Figure 1: System Architecture

6. Design Diagrams

6.1 Use Case Diagram

There is one actor, the **User**, who interacts with the system through the menu.

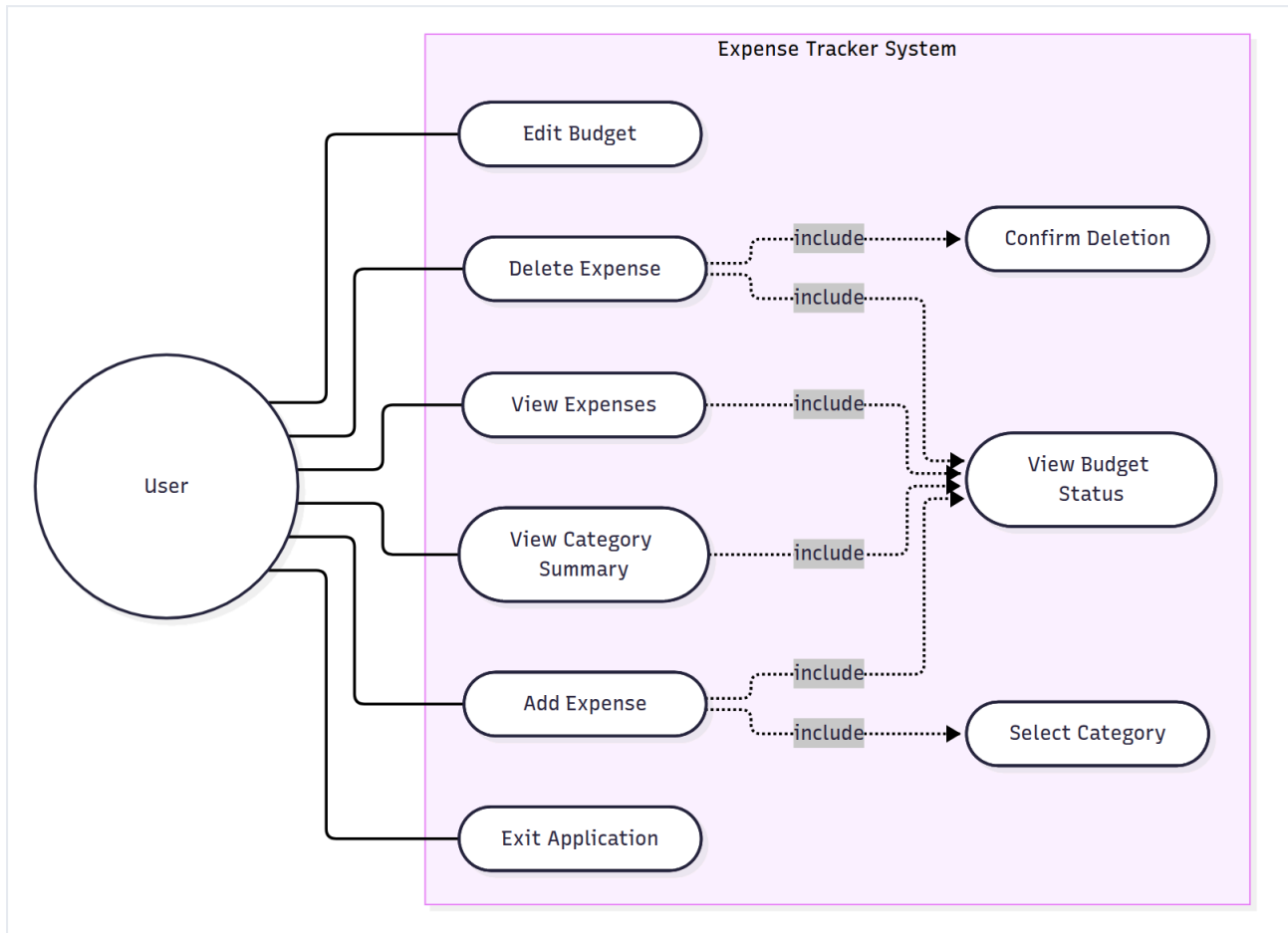


Figure 2: Use Case Diagram

6.2 Workflow Diagram

The program runs as a loop: show the menu, carry out the chosen action, and return to the menu until the user exits.

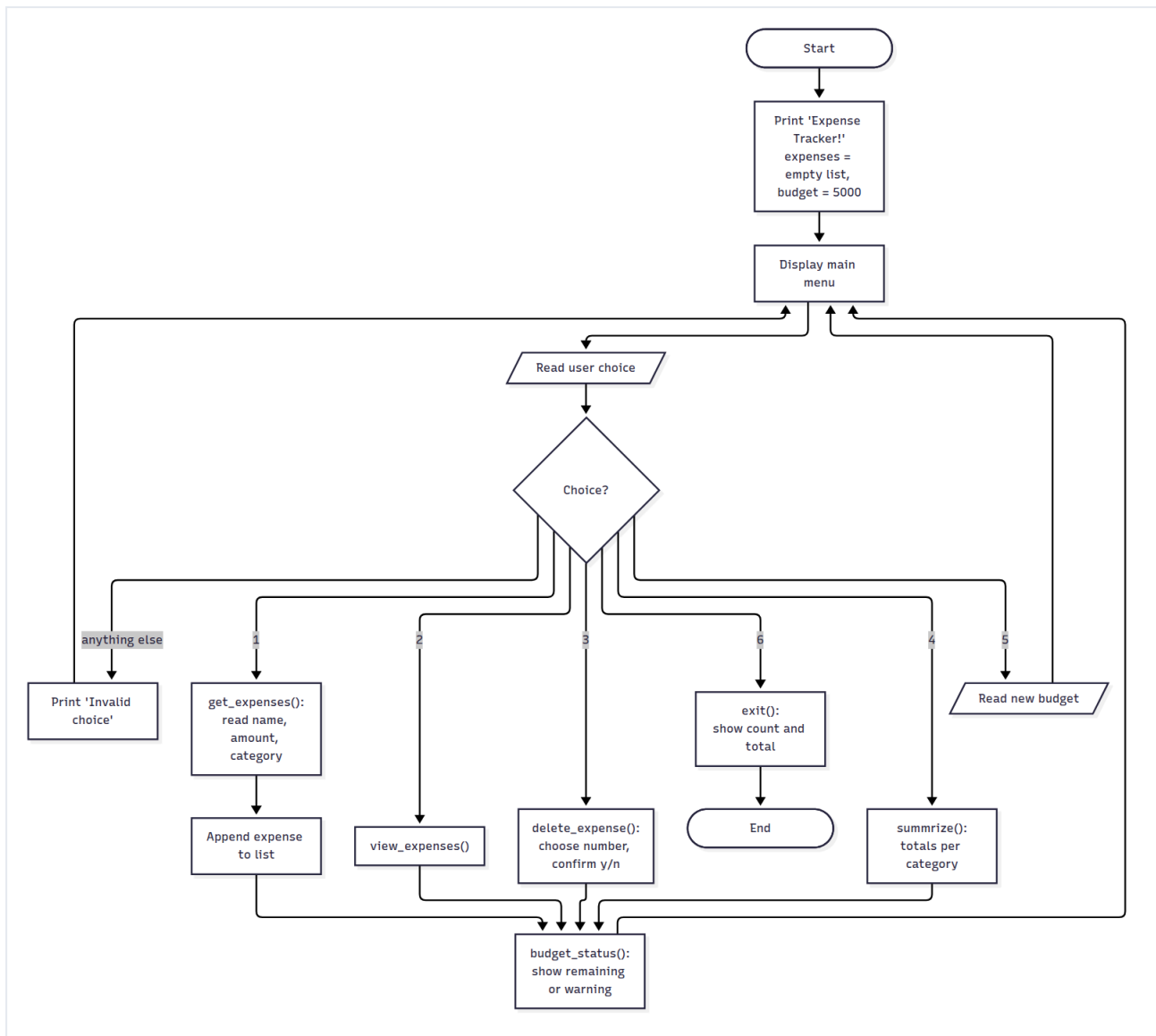


Figure 3: Workflow Diagram

6.3 Sequence Diagram: Adding an Expense

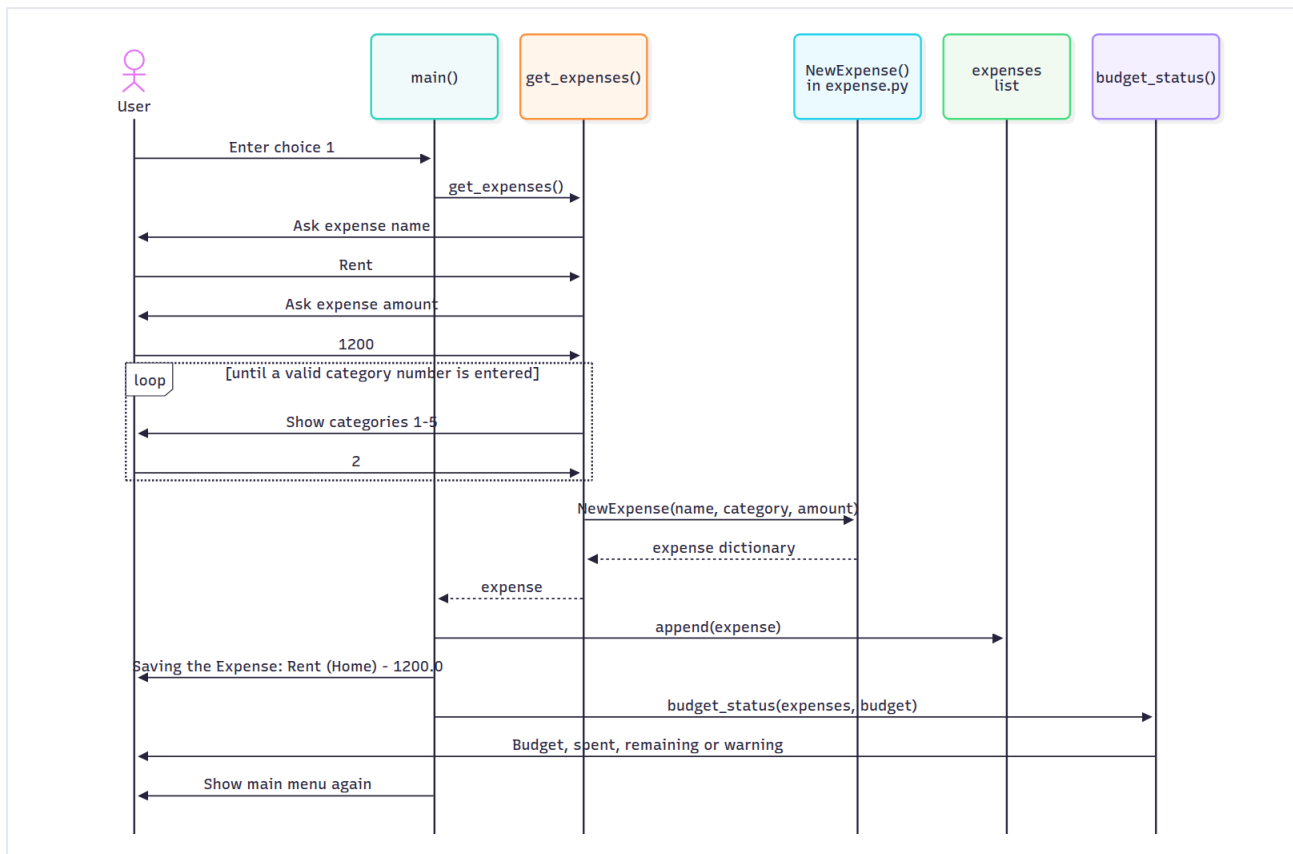


Figure 4: Sequence Diagram for Adding an Expense

6.4 Component Diagram

The project does not use classes, so a **component diagram** shows the two modules, their functions, and the expense record (a dictionary) that they work with.

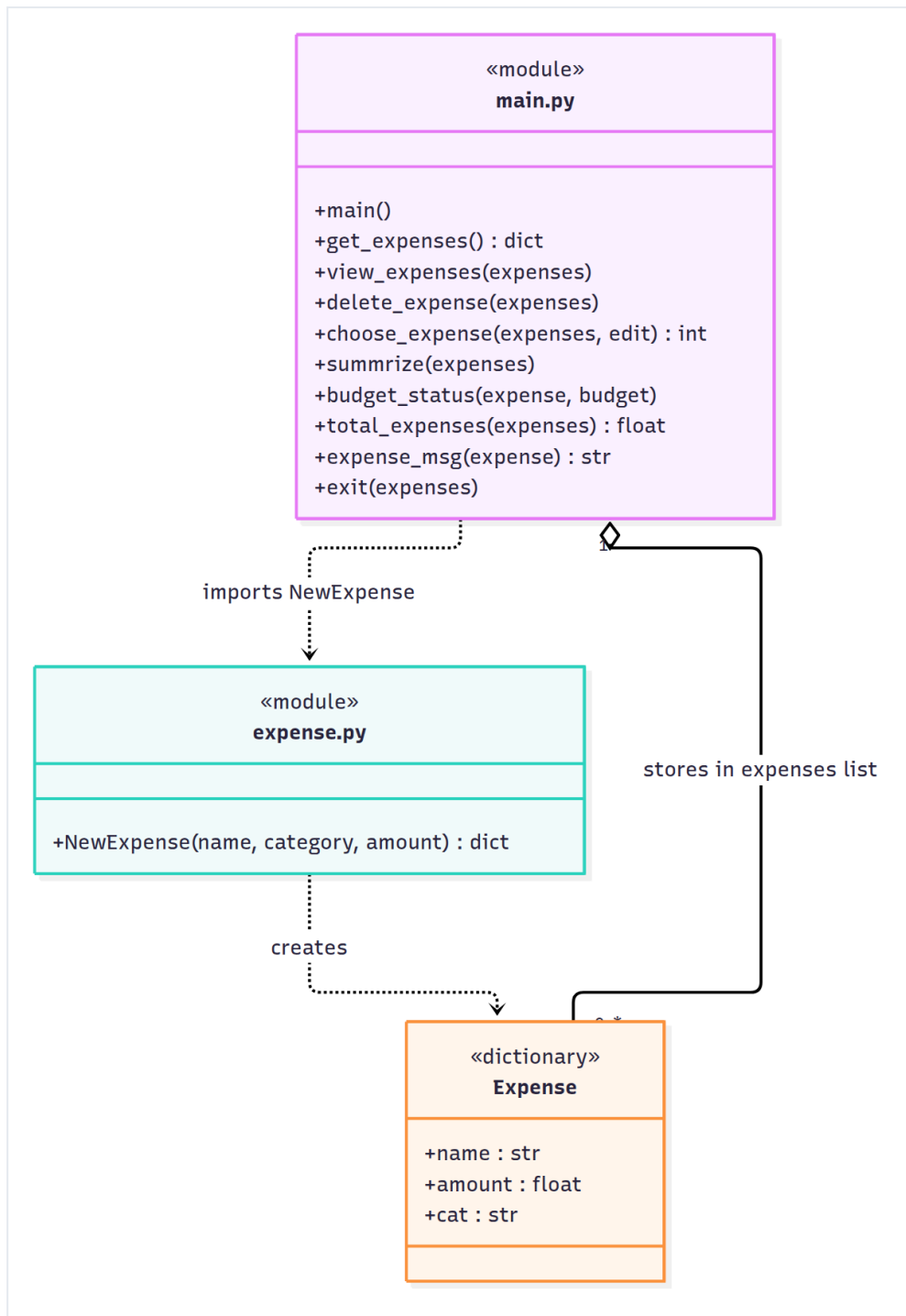


Figure 5: Component Diagram

6.5 ER Diagram / Storage Design

Not applicable. The application does not use a database or files. All data lives in memory while the program runs. The in-memory data structure is described below instead.

Expense record (Python dictionary)

Key	Type	Example	Description
name	str	"Rent"	What the money was spent on

Key	Type	Example	Description
amount	float	1200.0	Amount spent, in rupees
cat	str	"Home"	One of: Food, Home, Work, Entertainment, Misc

Program state (created in `main()`)

Variable	Type	Initial value	Description
expenses	list of expense dictionaries	<code>[]</code>	All expenses added in this session, in the order they were added
budget	int	5000	The spending limit in rupees

7. Design Decisions & Rationale

Decision	Alternatives considered	Reason for the choice
Command-line menu interface	GUI (Tkinter), web app	Keeps the focus on core programming logic; runs anywhere Python runs; a numbered menu is easy for any user.
Dictionary for each expense	A class with attributes, a tuple	A dictionary is simple, readable (<code>expense["amount"]</code>), and needs no extra class code. Named keys are clearer than tuple positions.
List to hold all expenses	Dictionary keyed by ID, database	Keeps the order expenses were added in, and the list position gives each expense a natural number for viewing and deleting.
In-memory storage only	CSV / JSON file, SQLite	Keeps the program simple with no file handling or file-corruption risk. It also means no financial data is left on the computer. Saving to a file is planned as a future enhancement.
Separate <code>expense.py</code> with <code>NewExpense()</code>	Build the dictionary inline in <code>main.py</code>	Keeps the structure of an expense in one place. If a field is added later (e.g. a date), only one function changes.
Fixed list of five categories	Free-text category	Picking a number avoids typos like "food" vs "Food", which would split the summary into two groups.
Budget with a default of Rs.5000	No budget until the user sets one	The user gets budget feedback right away. It can be changed at any time from the menu.
Delete asks for confirmation	Delete immediately	Prevents losing data by pressing the wrong number.
Helper functions <code>expense_msg()</code> and <code>total_expenses()</code>	Repeat the same code in each feature	Removes duplicate code, so formatting and total calculation stay consistent everywhere.
No external libraries	e.g. <code>tabulate</code> , <code>matplotlib</code>	No installation step, and every line of the program shows core Python concepts. The bar chart is drawn with <code>"#"</code> characters.

8. Implementation Details

8.1 Project Structure

```
Expense-tracker-/
├─ main.py           # Menu loop and all feature functions (166 lines)
├─ expense.py        # NewExpense(): creates an expense dictionary
├─ README.md         # Overview, features and how to run
├─ projectreport.md  # This report
├─ images/           # Diagrams and screenshots used in the report
├─ diagrams/         # Mermaid source for the diagrams
└─ .gitignore        # Keeps __pycache__/ out of Git
```

To run: `python main.py` (or `python3 main.py` on macOS / Linux). There is nothing to install.

8.2 Creating an Expense (`expense.py`)

```
def NewExpense(name, category, amount):
    expense = {
        "name": name,
        "amount": amount,
        "cat": category,
    }
    return expense
```

This function is the single place where the structure of an expense is defined.

8.3 Main Menu Loop (`main()`)

`main()` creates the program state and repeats the menu until the user chooses Exit. Each choice calls one feature function.

```
def main():
    print(f"Expense Tracker!")
    expenses = []
    budget = 5000
    ...
    while True:
        ...
        inp = input("Enter your choice [1 - 6]: ").strip()

        if inp == "1":
            expense = get_expenses()
            expenses.append(expense)
            print(f"Saving the Expense: {expense_msg(expense)}")
            budget_status(expenses, budget)
        elif inp == "2":
            view_expenses(expenses)
            budget_status(expenses, budget)
        ...
        elif inp == "6":
```

```

        exit(expenses)
    break
else:
    print("Invalid choice. Please try again!")

```

`.strip()` removes extra spaces, so an input like " 1 " still works. Comparing strings ("1") instead of converting to `int` means a letter typed at the menu cannot crash the program.

8.4 Adding an Expense (`get_expenses()`)

The function reads the name and amount, then loops until a valid category number is entered:

```

while True:
    print("Select a category: ")
    for i, category_name in enumerate(categories):
        print(f"{i + 1}. {category_name}")

    category_index = (int(input(f"Enter category number [1 - {len(categories)}]: ")) - 1)

    if category_index in range(len(categories)):
        new_expense = NewExpense(name=name, category=categories[category_index],
amount=amount)
        return new_expense
    else:
        print("Invalid category. please try again!")

```

`enumerate()` numbers the categories from 1 for the user, and `- 1` converts back to a list index starting at 0. `in range(...)` checks that the number is valid.

8.5 Category Summary (`summrize()`)

A dictionary groups the totals by category in one pass, then each category's share of the total is shown as a percentage and a bar:

```

amount_by_cat = {}
for data in expenses:
    key = data["cat"]
    if key in amount_by_cat:
        amount_by_cat[key] += data["amount"]
    else:
        amount_by_cat[key] = data["amount"]

total = total_expenses(expenses)

for cat, amount in amount_by_cat.items():
    percent = amount / total * 100
    bar = "#" * max(1, round(percent / 5))
    print(f" [] {cat}: {amount} ({percent}%) {bar}")

```

Each `#` stands for about 5% of total spending. `max(1, ...)` makes sure even a very small category gets at least one `#`.

8.6 Budget Status (`budget_status()`)

```
def budget_status(expense, budget=5000):
    total = total_expenses(expense)
    print(f"[] budget: Rs.{budget}, spent: {total}")

    if (total > budget):
        print(f"[] WARNING: you have exceeded the budget by {total - budget}")
    else:
        print(f"[] Remaining Budget: {budget - total}")
```

8.7 Deleting an Expense (`delete_expense()` and `choose_expense()`)

`choose_expense()` lists the expenses and returns the chosen list index, or `None` if the user presses Enter to cancel. `delete_expense()` then asks for confirmation and removes the item with `list.pop()`:

```
confirm = input(f"[] Delete {expense_msg(expenses[i])} ? (y/n)").lower().strip()

if confirm in ("y", "yes"):
    removed = expenses.pop(i)
    print(f"Deleted {expense_msg(removed)}")
else:
    print("Cancelled.")
```

`.lower()` means Y, y, YES and yes are all accepted.

8.8 Time Complexity

Operation	Complexity	Reason
Add expense	O(1)	<code>list.append()</code>
View expenses	O(n)	One loop over the list
Total spent	O(n)	One loop over the list
Category summary	O(n)	One pass to group, plus a loop over at most 5 categories
Delete expense	O(n)	<code>list.pop(i)</code> shifts the items after <code>i</code>

8.9 Version Control (Git and GitHub)

The project was developed in the GitHub repository <https://github.com/1divyansh/Expense-tracker->, with each step committed separately:

Commit	Date	Description
0b7d71b	28 Sep 2026	First working version built from the starter template
b72ea4f	28 Sep 2026	Rewrite as an in-memory app with a menu loop
bc7c1b7	29 Sep 2026	Add edit / delete, category summary, budget and input checks
af230ab	29 Sep 2026	Update <code>main.py</code> (current menu and features)

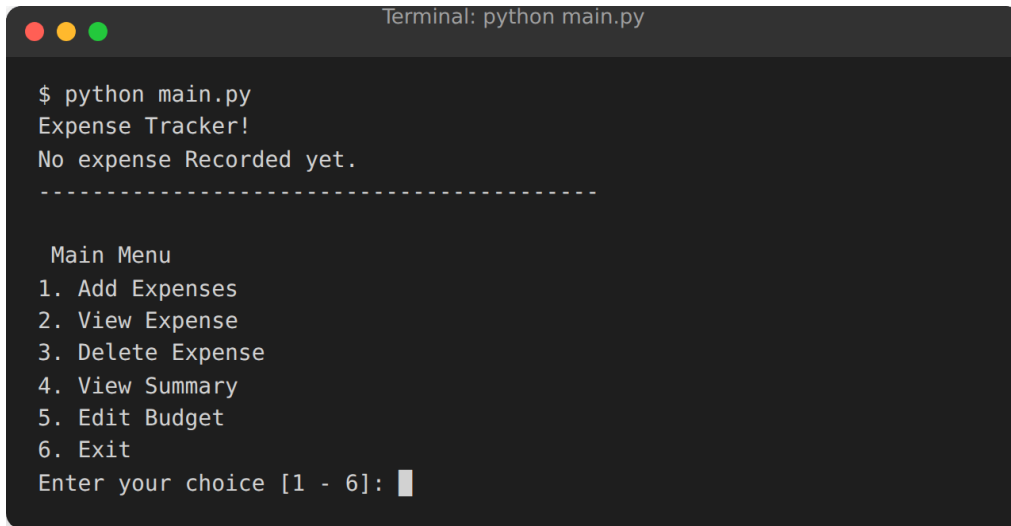
Commit	Date	Description
d10e1c1	29 Sep 2026	Create <code>expense.py</code>
f5da52d	29 Sep 2026	Add README
1d1300f	30 Sep 2026	Add project report with diagrams

(Update this table with `git log --oneline` if you add more commits.)

9. Screenshots / Results

Figures 6 to 11 and Figure 13 come from one continuous run of the program, so the amounts carry over from one figure to the next. Figure 12 comes from a separate run. The input typed for each figure is listed under it.

9.1 Program Start and Main Menu

A terminal window titled "Terminal: python main.py" with a dark background and light gray text. The output shows the program starting, displaying "Expense Tracker!" and "No expense Recorded yet." followed by a dashed line separator. Below the separator is the "Main Menu" with a numbered list of options: 1. Add Expenses, 2. View Expense, 3. Delete Expense, 4. View Summary, 5. Edit Budget, and 6. Exit. The prompt "Enter your choice [1 - 6]:" is followed by a white cursor block.

```
Terminal: python main.py

$ python main.py
Expense Tracker!
No expense Recorded yet.
-----

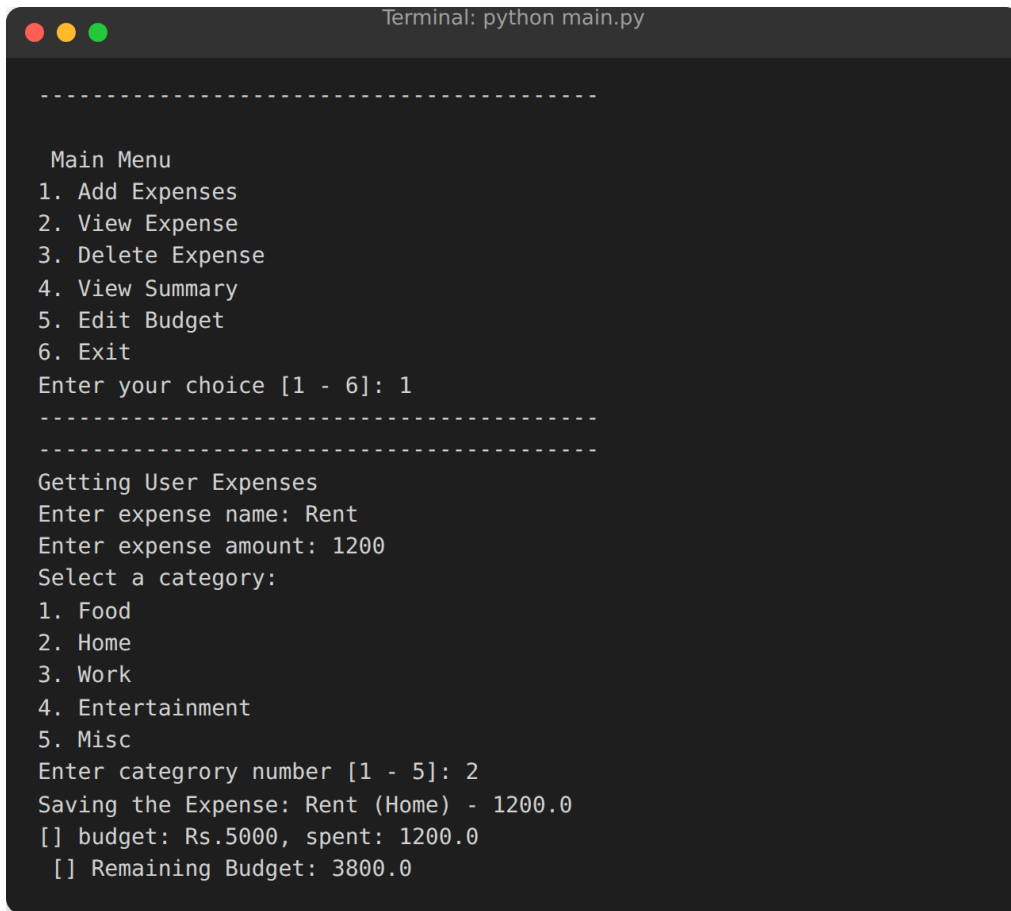
Main Menu
1. Add Expenses
2. View Expense
3. Delete Expense
4. View Summary
5. Edit Budget
6. Exit
Enter your choice [1 - 6]: █
```

Figure 6: Program start and main menu

Input: `python main.py`

Result: The program starts, reports that no expenses have been recorded yet, and waits for a menu choice.

9.2 Adding an Expense

A terminal window titled "Terminal: python main.py" with a dark background and light gray text. It shows the execution of a Python script. The script starts with a dashed line separator, followed by a "Main Menu" with six options: 1. Add Expenses, 2. View Expense, 3. Delete Expense, 4. View Summary, 5. Edit Budget, and 6. Exit. The user enters '1'. Another dashed line separator follows. The script then prompts for "Getting User Expenses", asking for the expense name (Rent), amount (1200), and category (1-5). The user enters '2' for the category. The script then displays the saved expense and the updated budget status.

```
Terminal: python main.py

-----

Main Menu
1. Add Expenses
2. View Expense
3. Delete Expense
4. View Summary
5. Edit Budget
6. Exit
Enter your choice [1 - 6]: 1
-----

-----

Getting User Expenses
Enter expense name: Rent
Enter expense amount: 1200
Select a category:
1. Food
2. Home
3. Work
4. Entertainment
5. Misc
Enter category number [1 - 5]: 2
Saving the Expense: Rent (Home) - 1200.0
[] budget: Rs.5000, spent: 1200.0
[] Remaining Budget: 3800.0
```

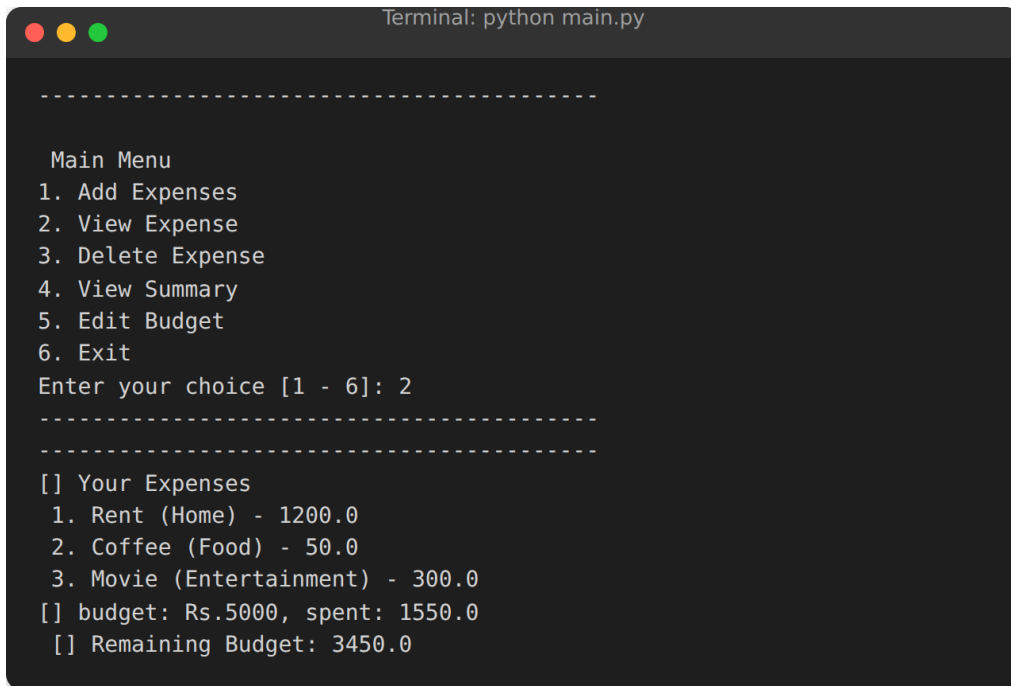
Figure 7: Adding an expense

Input: 1, name Rent, amount 1200, category 2

Result: The expense is saved as *Rent (Home)*. The budget status shows Rs.3800 remaining out of the default Rs.5000.

Two more expenses were then added the same way: Coffee, 50, Food and Movie, 300, Entertainment.

9.3 Viewing Expenses



```

Terminal: python main.py

-----

Main Menu
1. Add Expenses
2. View Expense
3. Delete Expense
4. View Summary
5. Edit Budget
6. Exit
Enter your choice [1 - 6]: 2
-----

[] Your Expenses
1. Rent (Home) - 1200.0
2. Coffee (Food) - 50.0
3. Movie (Entertainment) - 300.0
[] budget: Rs.5000, spent: 1550.0
[] Remaining Budget: 3450.0

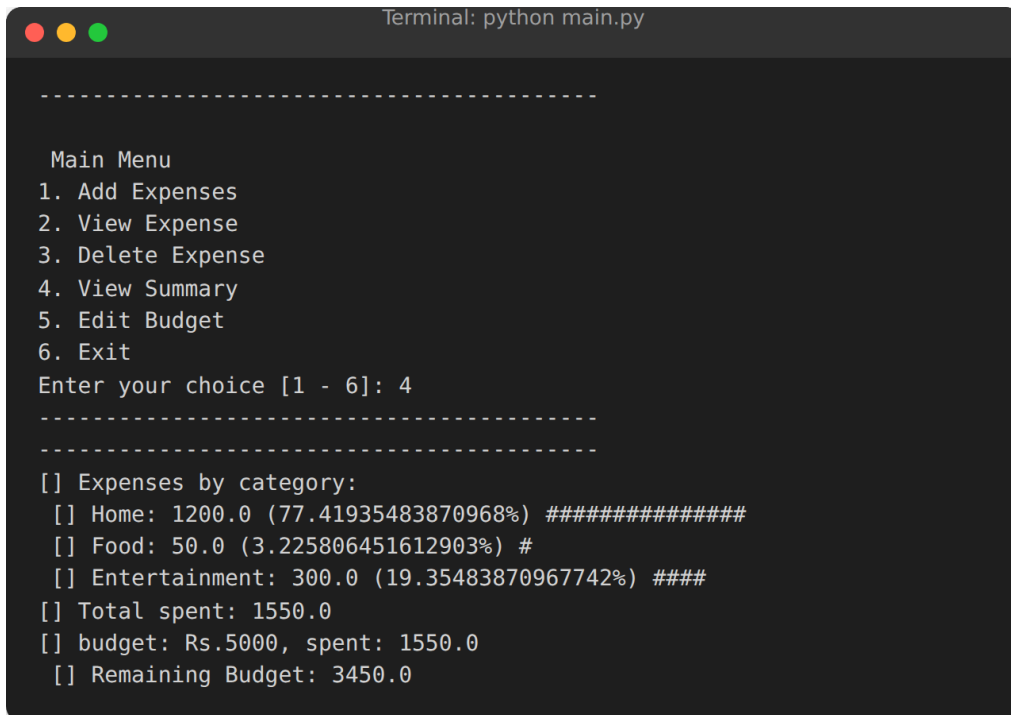
```

Figure 8: Viewing all expenses

Input: 2

Result: All three expenses are listed with their numbers. Total spent is 1550.0, leaving 3450.0 of the budget.

9.4 Category Summary



```

Terminal: python main.py

-----

Main Menu
1. Add Expenses
2. View Expense
3. Delete Expense
4. View Summary
5. Edit Budget
6. Exit
Enter your choice [1 - 6]: 4
-----

[] Expenses by category:
[] Home: 1200.0 (77.41935483870968%) #####
[] Food: 50.0 (3.225806451612903%) #
[] Entertainment: 300.0 (19.35483870967742%) ####
[] Total spent: 1550.0
[] budget: Rs.5000, spent: 1550.0
[] Remaining Budget: 3450.0

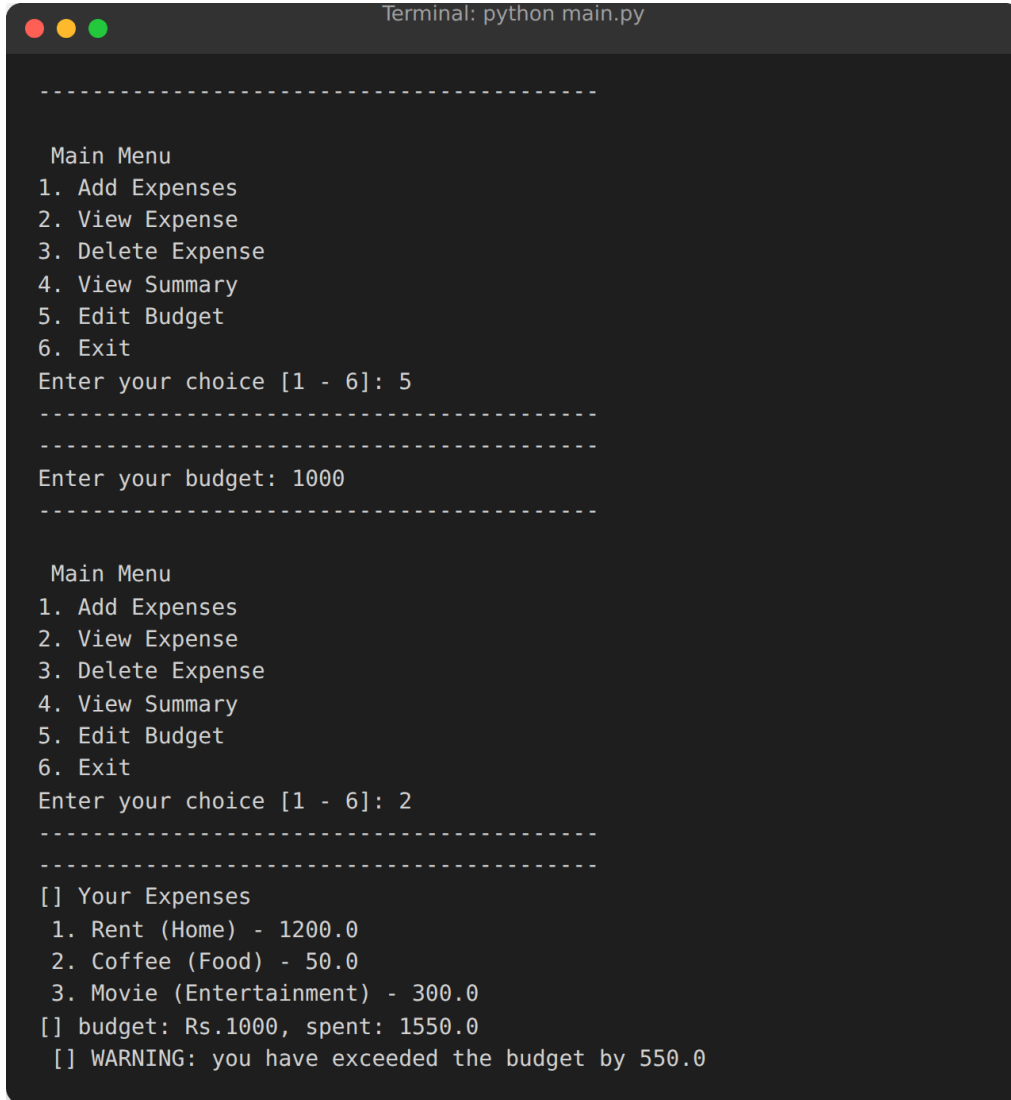
```

Figure 9: Category-wise summary

Input: 4

Result: Home takes 77.4% of spending, Entertainment 19.4% and Food 3.2%. Each # in the bar stands for about 5% of the total.

9.5 Budget Warning



```

Terminal: python main.py

-----

Main Menu
1. Add Expenses
2. View Expense
3. Delete Expense
4. View Summary
5. Edit Budget
6. Exit
Enter your choice [1 - 6]: 5
-----

Enter your budget: 1000
-----

Main Menu
1. Add Expenses
2. View Expense
3. Delete Expense
4. View Summary
5. Edit Budget
6. Exit
Enter your choice [1 - 6]: 2
-----

[] Your Expenses
1. Rent (Home) - 1200.0
2. Coffee (Food) - 50.0
3. Movie (Entertainment) - 300.0
[] budget: Rs.1000, spent: 1550.0
[] WARNING: you have exceeded the budget by 550.0

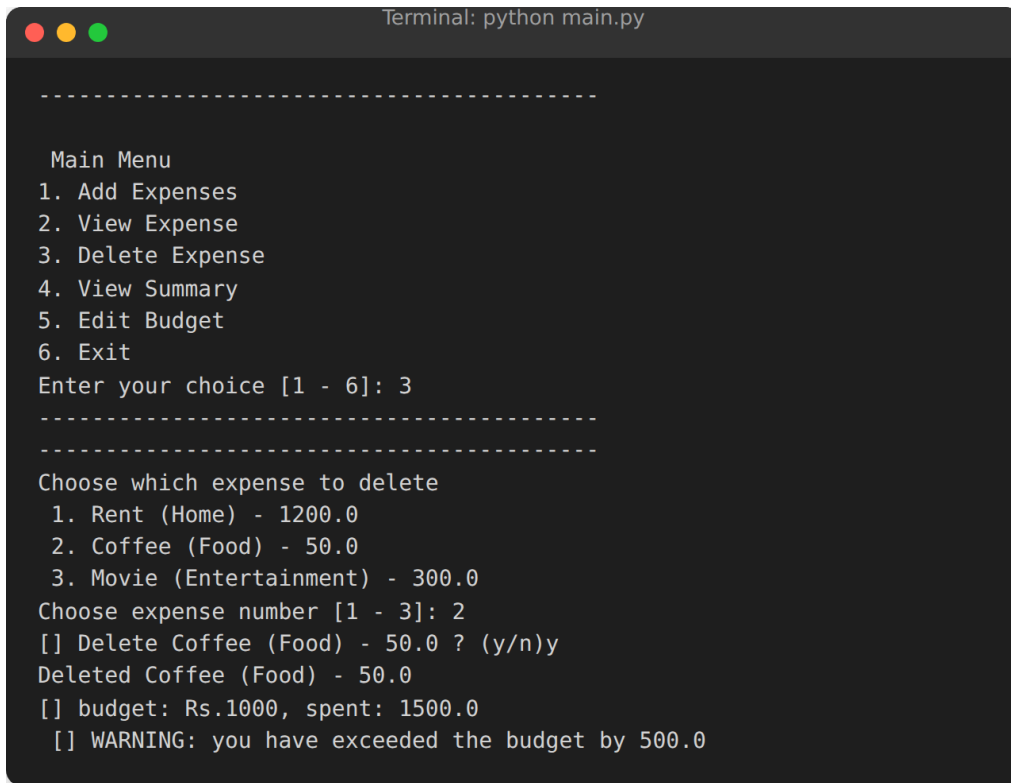
```

Figure 10: Budget exceeded warning

Input: 5, new budget 1000, then 2

Result: With the budget lowered to Rs.1000, spending of 1550.0 triggers the warning *you have exceeded the budget by 550.0*.

9.6 Deleting an Expense

A terminal window titled "Terminal: python main.py" with a dark background and light gray text. It shows the execution of a Python script. The script starts with a dashed line separator, followed by a "Main Menu" with six options: 1. Add Expenses, 2. View Expense, 3. Delete Expense, 4. View Summary, 5. Edit Budget, and 6. Exit. The user enters "3". Another dashed line separator follows. The script then displays "Choose which expense to delete" with three options: 1. Rent (Home) - 1200.0, 2. Coffee (Food) - 50.0, and 3. Movie (Entertainment) - 300.0. The user enters "2". The script then asks for confirmation: "[] Delete Coffee (Food) - 50.0 ? (y/n)y". It then displays "Deleted Coffee (Food) - 50.0", followed by "[] budget: Rs.1000, spent: 1500.0", and finally "[] WARNING: you have exceeded the budget by 500.0".

```
Terminal: python main.py

-----

Main Menu
1. Add Expenses
2. View Expense
3. Delete Expense
4. View Summary
5. Edit Budget
6. Exit
Enter your choice [1 - 6]: 3
-----

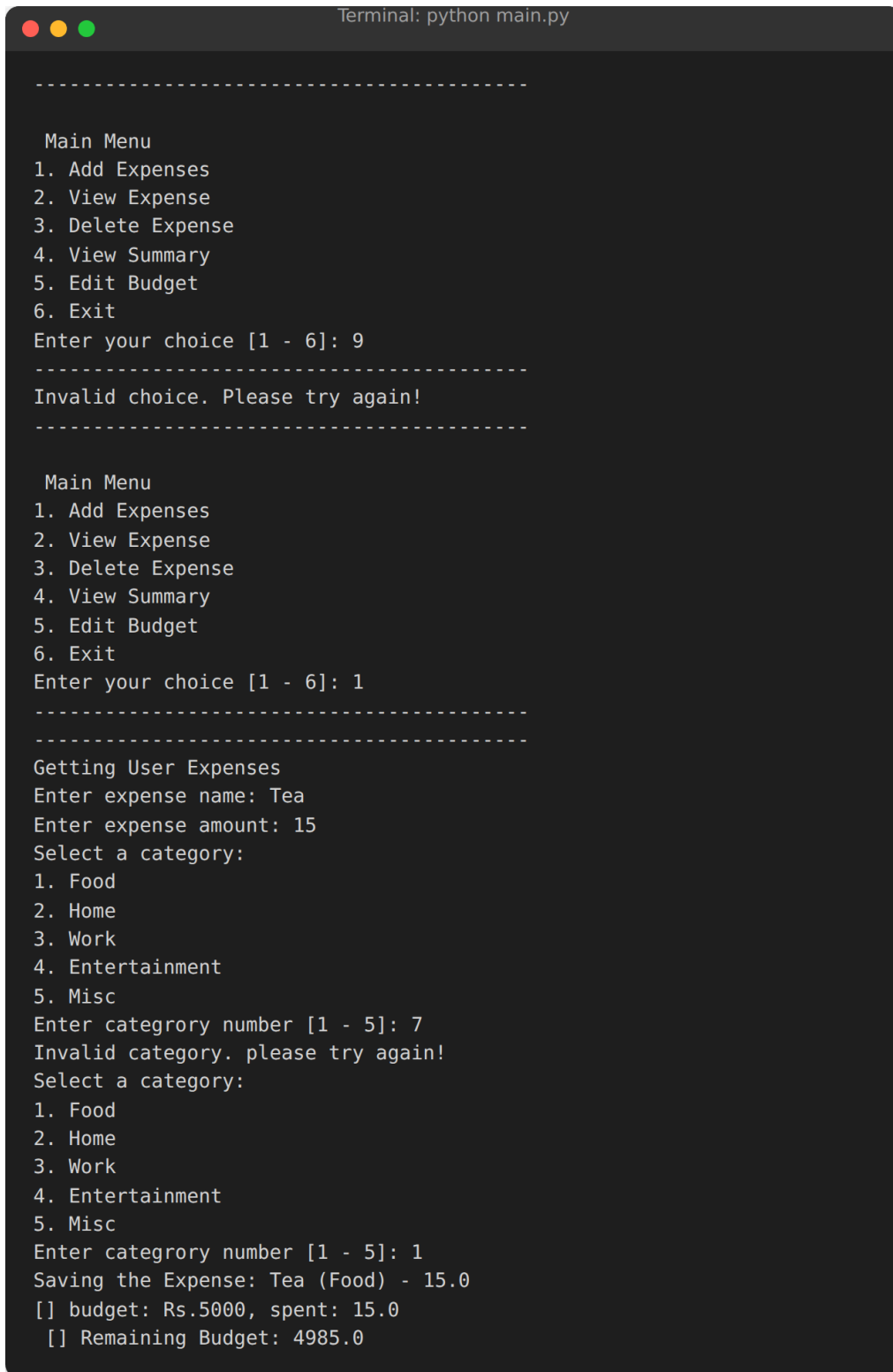
Choose which expense to delete
1. Rent (Home) - 1200.0
2. Coffee (Food) - 50.0
3. Movie (Entertainment) - 300.0
Choose expense number [1 - 3]: 2
[ ] Delete Coffee (Food) - 50.0 ? (y/n)y
Deleted Coffee (Food) - 50.0
[ ] budget: Rs.1000, spent: 1500.0
[ ] WARNING: you have exceeded the budget by 500.0
```

Figure 11: Deleting an expense with confirmation

Input: 3, expense number 2, then y

Result: After confirmation, *Coffee (Food)* is deleted. Total spent drops to 1500.0, which is still 500.0 over budget.

9.7 Handling Invalid Input



```

Terminal: python main.py

-----

Main Menu
1. Add Expenses
2. View Expense
3. Delete Expense
4. View Summary
5. Edit Budget
6. Exit
Enter your choice [1 - 6]: 9
-----
Invalid choice. Please try again!
-----

Main Menu
1. Add Expenses
2. View Expense
3. Delete Expense
4. View Summary
5. Edit Budget
6. Exit
Enter your choice [1 - 6]: 1
-----
-----
Getting User Expenses
Enter expense name: Tea
Enter expense amount: 15
Select a category:
1. Food
2. Home
3. Work
4. Entertainment
5. Misc
Enter category number [1 - 5]: 7
Invalid category. please try again!
Select a category:
1. Food
2. Home
3. Work
4. Entertainment
5. Misc
Enter category number [1 - 5]: 1
Saving the Expense: Tea (Food) - 15.0
[] budget: Rs.5000, spent: 15.0
[] Remaining Budget: 4985.0

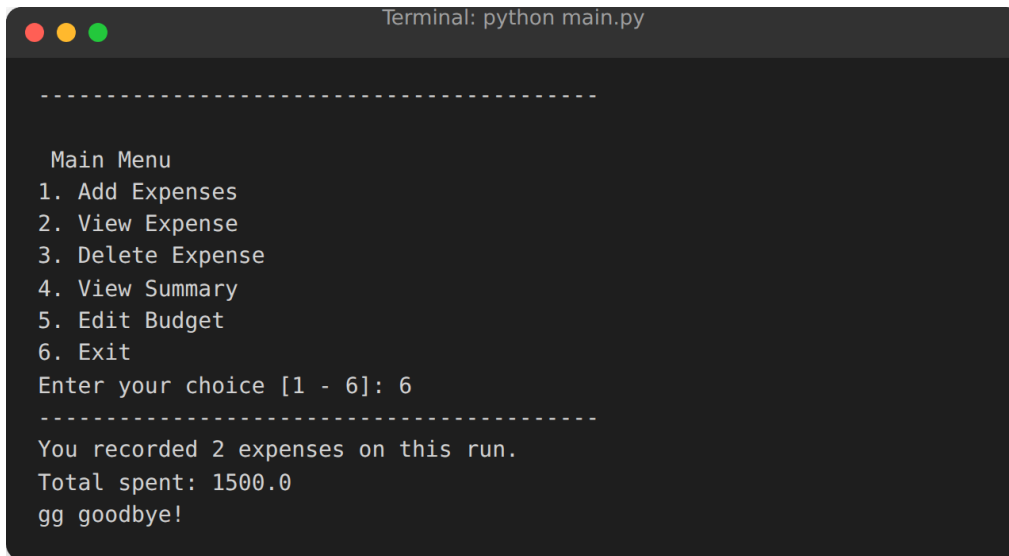
```

Figure 12: Handling invalid menu and category input

Input (separate run): 9, then 1, Tea, 15, category 7, then category 1

Result: The invalid menu choice and the invalid category number are both rejected with a message, and the program asks again instead of stopping. The expense is saved once a valid category is entered.

9.8 Exiting the Program



```

Terminal: python main.py

-----

Main Menu
1. Add Expenses
2. View Expense
3. Delete Expense
4. View Summary
5. Edit Budget
6. Exit
Enter your choice [1 - 6]: 6
-----

You recorded 2 expenses on this run.
Total spent: 1500.0
gg goodbye!

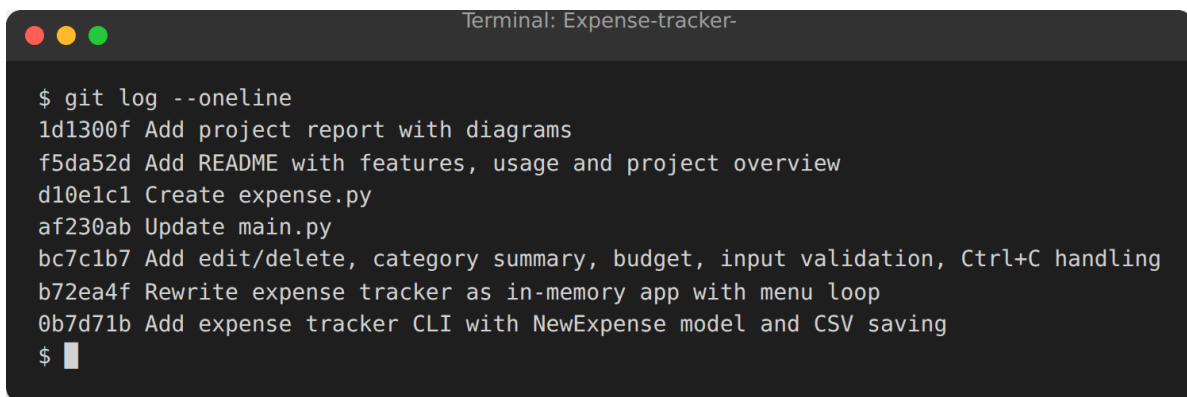
```

Figure 13: Exit summary

Input: 6

Result: The program shows how many expenses were recorded in this run and the total spent, then ends.

9.9 Version Control History



```

Terminal: Expense-tracker-

$ git log --oneline
1d1300f Add project report with diagrams
f5da52d Add README with features, usage and project overview
d10e1c1 Create expense.py
af230ab Update main.py
bc7c1b7 Add edit/delete, category summary, budget, input validation, Ctrl+C handling
b72ea4f Rewrite expense tracker as in-memory app with menu loop
0b7d71b Add expense tracker CLI with NewExpense model and CSV saving
$ 

```

Figure 14: Git commit history

Command: `git log --oneline` in the project folder

Result: The project's development history, with one commit for each step (see Section 8.9).

10. Testing Approach

10.1 Method

The application was tested with **manual black-box testing**: each feature was run with normal inputs, boundary inputs and invalid inputs, and the actual output was compared with the expected output. Test inputs were also fed to the program automatically through standard input (for example `printf '1\nRent\n1200\n2\n6\n' | python main.py`) so the same test could be repeated exactly after every change.

The tests cover three groups:

1. **Normal use**: every menu option with valid input
2. **Edge cases**: empty list, budget exceeded, cancelling a delete
3. **Invalid input**: wrong menu choice, wrong category, non-numeric and negative numbers

10.2 Test Cases and Results

ID	Module	Test input	Expected result	Actual result	Status
TC-01	Add	1, Rent, 1200, 2	Expense saved as Rent (Home)	Saving the Expense: Rent (Home) - 1200.0	PASS
TC-02	View	2 after adding two expenses	Numbered list of both	Both expenses listed as 1 and 2	PASS
TC-03	View	2 with no expenses	"No expenses" message	[] No expenses recorded	PASS
TC-04	Summary	4 with Home and Food expenses	Totals, % and bar for each	Both categories shown with % and # bar	PASS
TC-05	Summary	4 with no expenses	"No expenses" message	No expense Recorded yet.	PASS
TC-06	Budget	Budget 1000, add expense of 1200	Over-budget warning	WARNING: you have exceeded the budget by 200.0	PASS
TC-07	Budget	5, 2000, then 2	Remaining budget is 2000	Remaining Budget: 2000	PASS
TC-08	Menu	9	Error, menu shown again	Invalid choice. Please try again!	PASS
TC-09	Add	Category 9, then 1	Error, then asks again	Invalid category. please try again!, then saved	PASS
TC-10	Delete	Two expenses, 3, 2, y	Expense 2 removed	Only expense 1 remains	PASS
TC-11	Delete	Two expenses, 3, 2, n	Nothing removed	Cancelled., both remain	PASS
TC-12	Delete	3 with no expenses	"No expenses" message	There is no expense Recorded	PASS
TC-13	Exit	6	Count, total and goodbye	You recorded 2 expenses on this run. ... gg goodbye!	PASS

ID	Module	Test input	Expected result	Actual result	Status
TC-14	Delete	Two expenses, 3, 1	Asks to confirm, deletes expense 1	Returns silently, nothing deleted	FAIL
TC-15	Delete	3, 0, y	Error: invalid number	Deletes the last expense	FAIL
TC-16	Delete	3, abc	Error, asks again	Program stops with <code>ValueError</code>	FAIL
TC-17	Add	Amount abc	Error, asks again	Program stops with <code>ValueError</code>	FAIL
TC-18	Add	Category x	Error, asks again	Program stops with <code>ValueError</code>	FAIL
TC-19	Add	Amount -50	Rejected: amount must be > 0	Accepted; total spent becomes negative	FAIL
TC-20	Budget	5, 1500.50	Accept decimal budget or show error	Program stops with <code>ValueError</code>	FAIL

10.3 Summary of Results

- **13 of 20 tests passed.** All the main features work correctly with valid input.
- **7 tests failed**, and they all come from **missing input validation**, not from wrong calculations:
 - `int()` and `float()` raise `ValueError` when the text is not a number (TC-16, 17, 18, 20).
 - `choose_expense()` does not check that the number is between 1 and the number of expenses. The input 0 becomes index -1, which Python treats as the last item (TC-15).
 - `delete_expense()` checks `if not i:` to detect "cancelled". Since 0 is treated as False in Python, choosing expense 1 (index 0) is treated as a cancel (TC-14). Checking `if i is None:` fixes this.
 - Amounts are not checked to be greater than zero (TC-19).

The fixes for these cases are listed in Section 13.

11. Challenges Faced

1. **Bugs in the starter template.** The given template defined `save_expense()` twice, so the second definition silently replaced the first. It also called the function without the argument it needed, which causes a `TypeError`. Understanding *why* Python allowed the duplicate definition was an important first step.
2. **Choosing how to store data.** Saving expenses to a CSV file was considered, but in-memory storage was chosen to keep the project focused on data structures and program logic. The trade-off is that data is lost when the program closes.
3. **Handling invalid input.** Converting input with `int()` and `float()` crashes the program if the user types letters. Testing showed which inputs break the program (Section 10), and handling every case properly with `try/except` is ongoing work.
4. **The "0 is False" bug.** Using `if not i:` to check for "no selection" also treats index `0` as no selection, so the first expense could not be deleted. This showed the difference between `None` and `0` in Python's truthiness rules.
5. **Keeping Git in sync.** Changes were made both on GitHub and on a local copy at the same time, so a push was rejected because the two histories had diverged. This was solved by fetching the remote changes and rebasing local work on top of them.
6. **Readable output in a text-only interface.** Showing a summary without graphics libraries led to the `#` bar chart, and to separator lines and `[]` markers to keep sections easy to tell apart.

12. Learnings & Key Takeaways

- **Breaking a problem into functions** makes a program easier to write, test and change. Each menu option maps to one function.
- **Choosing the right data structure matters.** A list keeps order and gives each item a number; a dictionary gives readable named fields and makes grouping by category simple.
- **Never trust user input.** Almost every failure found in testing came from input that was not checked. Validation deserves as much attention as the main features.
- **Python details can cause subtle bugs**, for example truthiness (`0` counts as `False`), negative list indexes (`-1` is the last item), and a later function definition silently replacing an earlier one.
- **Testing with a written table of cases** finds bugs that normal use does not, especially edge cases like deleting the first item.
- **Git** keeps a history of every change, makes it safe to try new ideas, and requires care when the same branch is changed in two places.
- **Design before coding:** writing requirements and drawing diagrams made the structure of the program clear before and during implementation.

13. Future Enhancements

Fixes (high priority, from testing):

1. Wrap every number input in `try/except ValueError` and ask again instead of crashing.
2. In `choose_expense()`, accept only numbers from 1 to the number of expenses.
3. In `delete_expense()`, change `if not i:` to `if i is None:` so the first expense can be deleted.
4. Reject empty names and amounts of zero or less, and allow decimal budgets.
5. Round percentages to one decimal place and format amounts as `Rs.1,200.00`.

New features:

1. **Edit an expense** (change its name, amount or category).
2. **Save data** to a CSV or JSON file so expenses are kept between runs.
3. **Dates** on each expense, with daily and monthly reports.
4. **Custom categories** added by the user.
5. **Sorting and filtering:** by amount, by category, or by date.
6. **Automated unit tests** (for example with Python's `unittest` module) for `total_expenses()`, `summrize()` and the input checks.
7. **Graphical interface** using Tkinter, with charts of spending by category.
8. Exit cleanly when the user presses **Ctrl+C**.

14. References

1. Python Software Foundation. *The Python Tutorial*. <https://docs.python.org/3/tutorial/>
2. Python Software Foundation. *Data Structures (lists and dictionaries)*.
<https://docs.python.org/3/tutorial/datastructures.html>
3. Python Software Foundation. *Built-in Functions: `input()`, `int()`, `float()`, `enumerate()`, `round()`*.
<https://docs.python.org/3/library/functions.html>
4. Python Software Foundation. *Errors and Exceptions*. <https://docs.python.org/3/tutorial/errors.html>
5. Python Software Foundation. *Formatted string literals (f-strings)*.
<https://docs.python.org/3/tutorial/inputoutput.html#formatted-string-literals>
6. van Rossum, G., Warsaw, B., & Coghlan, N. *PEP 8: Style Guide for Python Code*.
<https://peps.python.org/pep-0008/>
7. Chacon, S., & Straub, B. *Pro Git* (2nd ed.). Apress. <https://git-scm.com/book/en/v2>
8. GitHub Docs. *About commits and branches*. <https://docs.github.com/en/pull-requests/committing-changes-to-your-project>
9. Mermaid. *Diagram syntax documentation*. <https://mermaid.js.org/intro/>
10. VITyarthi. *Build Your Own Project: General Project Instructions & Submission Guidelines*.